

# Clase 169 — Ofuscación de payloads y bypass de AMSI

Parte: **7 — Red Team y operaciones ofensivas** · Fuente: *RTFM v2 (Clark) / Microsoft AMSI documentation* 🕒 Duración estimada: **110 min** · Nivel: **Experto**

## 🎯 Objetivo

Comprender AMSI (Antimalware Scan Interface) y las técnicas de ofuscación de payloads que lo evaden, entendiendo el mecanismo en profundidad. El alumno verá cómo AMSI inspecciona scripts en memoria (PowerShell, VBA, JScript), por qué la ofuscación simple ya no basta, y las estrategias de bypass responsables, con foco en cómo se detectan.

## 📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** el funcionamiento de AMSI y qué motores lo consumen.
2. **Aplicar** ofuscación a scripts y evaluar su efectividad.
3. **Describir** las categorías de bypass de AMSI (memory patching, providers, downgrade).
4. **Reconocer** por qué un bypass "público" deja de funcionar.
5. **Detectar** los IOCs que deja un intento de bypass.

## 📖 Temas

| # | Tema                                  | Por qué importa                    |
|---|---------------------------------------|------------------------------------|
| 1 | Qué es AMSI                           | Inspección de contenido en runtime |
| 2 | Integraciones (PowerShell, VBA, .NET) | AMSI cubre múltiples motores       |
| 3 | Ofuscación de scripts                 | Rompe firmas estáticas             |
| 4 | AMSI memory patch                     | Neutraliza el scan en el proceso   |
| 5 | Downgrade y forzar errores            | Alternativas al patch              |
| 6 | Detección de bypass                   | Cómo lo ve el Blue Team            |
| 7 | Ofuscación de binarios                | Más allá de scripts                |

## 📖 Definiciones y características

- **AMSI:** interfaz de Windows que permite a los antivirus escanear contenido (scripts, macros) justo antes de ejecutarse. Característica: ve el código ya "desofuscado" en memoria.
- **Ofuscación:** transformar el código para romper firmas manteniendo la funcionalidad. Característica: derrota firmas, no comportamiento.

- **AMSI bypass (memory patch):** modificar en memoria la función que devuelve el resultado del scan (ej. `AmsiScanBuffer`) para que siempre diga "limpio". Característica: muy usado y muy detectado.
- **Downgrade:** forzar el uso de PowerShell v2 (sin AMSI). Característica: evita AMSI si la v2 está disponible.
- **Provider tampering:** manipular el proveedor de AMSI registrado. Característica: técnica más sigilosa pero compleja.
- **String obfuscation / encoding:** dividir y codificar cadenas sospechosas. Característica: primer nivel de evasión estática.

## Herramientas y preparación

- Windows con PowerShell 5+ y Microsoft Defender activo (en VM de laboratorio).
- Herramientas de ofuscación de estudio (Invoke-Obfuscation como referencia histórica) y editores de scripts.
- Sysmon + Script Block Logging (Parte 8) para observar la detección.
- El C2 previo para probar la entrega de scripts evadidos.

 **Solo laboratorio.** Estas técnicas se practican en máquinas propias para entender AMSI y escribir mejores detecciones. Muchos bypass son públicos y por eso mismo ya están firmados. Nunca uses esto fuera de un engagement autorizado.

## Laboratorio guiado

1. **Provoca una detección.** En PowerShell del lab, ejecuta una cadena claramente maliciosa conocida (ej. la firma de prueba de AMSI) y observa que Defender la bloquea.
2. **Entiende dónde escanea AMSI.** Comprueba que la detección ocurre en memoria al ejecutar, no al guardar el archivo: AMSI ve el contenido tras la desofuscación superficial.
3. **Ofuscación básica.** Divide y concatena las cadenas sospechosas; observa si la firma estática deja de coincidir y por qué AMSI puede seguir viéndolo en runtime.
4. **Estudia el memory patch.** Analiza conceptualmente cómo un bypass parchea el retorno de `AmsiScanBuffer`; ejecútalo en el lab y verifica que un script antes bloqueado ahora corre.
5. **Downgrade (si aplica).** Prueba invocar `powershell -version 2` en un sistema que lo permita y comprueba la ausencia de AMSI (documenta que en sistemas endurecidos v2 no está).
6. **Observa la detección.** Con Script Block Logging activo, revisa cómo el Blue Team ve el bypass (patrones de reflexión, `[Ref].Assembly`, strings característicos).
7. **Ofuscación de binarios.** Compara cómo un packer/cifrado cambia el hash del payload C2 sin cambiar su comportamiento, y por qué el EDR aún lo detecta.

## Ejercicios

1. Explica con tus palabras qué ventaja tiene AMSI sobre el escaneo de archivos en disco.
2. Ofusca un script de prueba y evalúa si evade la firma estática.
3. Describe paso a paso la lógica de un AMSI memory patch.
4. Explica por qué el downgrade a PS v2 evade AMSI y cómo mitigarlo.
5. Enumera 4 IOCs que deja un intento de bypass en los logs.
6. Investiga por qué los bypass públicos "caducan" rápido.

## Reto verificable

En tu laboratorio, toma un script que Defender bloquea y consigue ejecutarlo aplicando una técnica de bypass de AMSI, documentando **cómo el Blue Team lo detectaría**. **Criterio de aceptación:** demuestras la ejecución del script tras el bypass (antes bloqueado) y entregas una regla o lista de indicadores (Script Block Logging, cadenas, comportamiento) con la que un defensor detectaría tu técnica. Todo en tu VM.

## Errores comunes

| Síntoma / mensaje                 | Causa y cómo arreglar                                       |
|-----------------------------------|-------------------------------------------------------------|
| El bypass público no funciona     | Ya está firmado; entiende el mecanismo y adáptalo           |
| Ofusco pero AMSI sigue detectando | AMSI ve el runtime; la ofuscación estática no basta         |
| Downgrade falla                   | PS v2 deshabilitado (buen hardening); no hay atajo          |
| El patch bloquea PowerShell       | Offset/versión incorrectos; verifica la firma de la función |
| Script Block Logging te delata    | Es telemetría rica; asume que el bypass es visible          |

## Preguntas frecuentes

 **¿Ofuscar es suficiente para evadir AMSI?** No. AMSI inspecciona el contenido en memoria tras la desofuscación superficial, así que la ofuscación estática por sí sola raramente basta contra motores actualizados.

 **¿Por qué los bypass de GitHub dejan de funcionar?** Porque en cuanto se publican, Microsoft los firma. La comprensión del mecanismo es lo duradero; el snippet concreto, no.

 **¿Deshabilitar AMSI es lo mismo que evadirlo?** No exactamente: evadir es lograr que no bloquee tu contenido concreto; los memory patches efectivamente lo neutralizan en el proceso, lo cual es muy detectable.

## Referencias

- Microsoft — *Antimalware Scan Interface (AMSI)*. <https://learn.microsoft.com/windows/win32/amsi/>
- MITRE ATT&CK — *Impair Defenses ( T1562 )*. <https://attack.mitre.org/techniques/T1562/>
- Red Canary / research sobre detección de AMSI bypass.
- Clark, B. — *RTFM: Red Team Field Manual v2*.

## Siguiente clase

Clase 170 - Active Directory: enumeracion